

NETSCOUT

Build & Packaging Guide

How to turn NetScout into a standalone app for Windows, macOS & Linux

BUILDING NETSCOUT AS A STANDALONE APP (no Python needed on the target machine)

WHY DOES THIS HAVE TO BE DONE SEPARATELY PER OPERATING SYSTEM?

The tool that does this is called PyInstaller. It packs the Python code, together with everything it needs, into a program of its own. But it cannot "cross-build" for a different operating system - a Windows .exe has to be built on a Windows machine, a macOS version on a Mac, a Linux version on a Linux machine. That is a limit of PyInstaller itself, not of NetScout.

The machine that does the BUILDING needs Python for that, once. The finished result needs NO Python - and the build scripts now PROVE that before they finish (see "THE AUTOMATIC CHECK" below).

DOWNLOADS (only needed on the BUILD machine)

```
Python ............ https://www.python.org/downloads/
   Windows ........ https://www.python.org/downloads/windows/
   macOS .......... https://www.python.org/downloads/macos/
   Linux .......... use your package manager (python3, python3-tk, python3-pip)
bleak (Bluetooth) . https://pypi.org/project/bleak/
                    docs: https://bleak.readthedocs.io/
                    (the build scripts install it for you - this is just for reference)
Pillow (camera) ... https://pypi.org/project/pillow/
                    (also installed by the build scripts; shows the IP CAMERA picture)
PyInstaller ....... https://pyinstaller.org/
                    (also installed by the build scripts automatically)
Microsoft Visual C++ Redistributable (rarely needed, see troubleshooting):
                    https://aka.ms/vs/17/release/vc_redist.x64.exe
```

FILES YOU NEED (put them all in one folder):

```
netscout.py             - the engine
netscout_gui.py         - the interface
netscout.spec           - build recipe for PyInstaller (same for all 3 systems)
netscout.ico            - app icon, Windows
netscout.icns           - app icon, macOS
netscout.png            - app icon, Linux
build_windows.bat      \
build_macos.sh          |-  use whichever script matches your system
build_linux.sh         /
```

STEPS:

```
Windows:
  1. Install Python FROM PYTHON.ORG (link above) - NOT the Microsoft Store
     version (see "WHY IT USED TO NEED PYTHON ON SOME PCS"). On the first
     installer screen tick "Add python.exe to PATH"; leave "tcl/tk and
     IDLE" ticked.
  2. Put all the files above in one folder.
  3. Double-click build_windows.bat.
  4. Results, in the dist folder:
       NetScout.exe          - one single file. Hand this out.
       NetScout-Folder.zip   - the same program as a folder. Hand this out
                               for PCs where the single file won't open
                               (strict antivirus / company laptops).
     Plus a "NetScout" shortcut on your Desktop.
  5. Copy NetScout.exe (or the zip) to as many Windows PCs as you like.

  NEVER hand out the .py files, and never anything from the "build"
  folder - those are the ones that need Python. (The script deletes the
  build folder now, so it can't happen by accident.)

macOS:
  1. Put all the files above in one folder.
  2. Terminal, in that folder:  bash build_macos.sh
  3. Result: dist/NetScout.app - offers to copy it into /Applications.
  4. Copy it to other Macs with the SAME KIND OF CHIP (Apple Silicon /
     M-series and Intel don't mix - when in doubt, build once on each).

Linux:
  1. Put all the files above in one folder.
  2. Terminal:  bash build_linux.sh
  3. Result: dist/NetScout, plus a menu entry and a Desktop entry. On GNOME a
     Desktop entry may need a one-time right-click -> "Allow Launching".
  4. Copy the binary to other Linux machines - most reliable on machines
     with a distro release as new as, or OLDER than, the build machine's.
```

WHY IT USED TO NEED PYTHON ON SOME PCS - AND WHAT WAS FIXED

The report was: the .exe works on some Windows PCs, but others ask for Python. A PyInstaller program normally carries its own Python inside, so this always means something it needed was NOT packed in, and those PCs that "worked" were silently borrowing it from their own Python install. The causes, and what the new build does about each:

```
1. Tcl/Tk (the library that draws the window) not packed in.
   Happens when the build machine uses the MICROSOFT STORE Python, or a
   Python whose Tcl/Tk is broken. Fixed: netscout.spec now forces Tcl/Tk
   into the build, and build_windows.bat REFUSES to build with Store Python
   or with a Python whose Tcl/Tk doesn't work - with a message telling you
   exactly what to install.

2. Antivirus / company security blocking the single .exe.
   The single-file .exe unpacks itself into the Temp folder every time it
   starts. Strict antivirus sometimes blocks that, and the error then says
   "Failed to load Python DLL" - which looks like "install Python" but
   isn't. Fixed: the script now also builds NetScout-Folder.zip, which
   never unpacks anything and runs on those PCs.

3. 32-bit Windows, or an ARM64 build.
   A program built with 64-bit Python does not run on old 32-bit Windows,
   and one built on an ARM64 (Snapdragon) laptop does not run on normal
   PCs. Fixed: the script now prints which kind it's building and warns.
   (NetScout needs 64-bit Windows 10 or 11 - Python itself no longer
   supports Windows 7/8.)

4. Someone copied the wrong file (a .py, or the half-built .exe inside the
   "build" folder). Fixed: the build folder is deleted after building, and
   the finished message lists exactly which files to hand out.
```

THE AUTOMATIC CHECK

After building, every build script starts the finished program with Python HIDDEN from it (Python removed from PATH, every Python/Tcl setting cleared) and asks it to open and close a hidden window (NetScout.exe --selftest). If the program secretly leaned on the installed Python, it fails right there, on the build machine, with a clear error - instead of later on someone else's laptop. A build that says "DONE - and verified to run without Python" is one you can hand out.

BLUETOOTH IN THE BUILT APP:

Only works if bleak was installed at build time - the scripts do that automatically, as a SEPARATE, OPTIONAL step. If it fails (no ready-made bleak package exists for your Python build, say), the scripts warn and build anyway: everything except Bluetooth works, and the app says plainly that Bluetooth isn't available in that copy. pip is told to use ready-made packages only (--only-binary) so it never tries to compile bleak's Windows "winrt" parts from source, which needs a full MSVC toolchain and can grind for minutes before crashing.

THE CAMERA PICTURE IN THE BUILT APP: Same idea as bleak: Pillow is installed as a separate, optional step. Without it the built app's IP CAMERA still logs in and steers cameras (W/A/S/D) - it just can't draw the picture, and says so.

USE A NORMAL PYTHON, NOT A "FREE-THREADED" ONE:

Python 3.13 also comes in an experimental "free-threaded" flavour (3.13t). Hardly any package has a ready-made build for it yet, bleak included. Use a plain Python 3.11, 3.12 or 3.13 from python.org. build_windows.bat warns if it sees the free-threaded one.

TROUBLESHOOTING

```
"No working Python installation was found"
    Install Python from python.org (link above), tick "Add python.exe to
    PATH". If Windows opens the Microsoft Store instead: Settings > Apps >
    Advanced app settings > App execution aliases, turn OFF python.exe and
    python3.exe.
"This is the Microsoft Store version of Python"
    Install the python.org version, then run the script again.
"This Python has no working Tcl/Tk"
    Run the python.org installer again > Modify > tick "tcl/tk and IDLE".
    Linux: sudo apt install python3-tk (Debian/Ubuntu),
           sudo dnf install python3-tkinter (Fedora).
"The built NetScout.exe did NOT start without Python"
    Check your antivirus quarantine, restart, and build again. To see the
    exact reason: set console=True in netscout.spec, build, and run
    dist\NetScout.exe from a Command Prompt.
On a target PC: "Failed to load Python DLL"
    Use NetScout-Folder.zip on that PC. If even that fails, install the
    Microsoft Visual C++ Redistributable (link above) on it.
On a target PC: "This app can't run on your PC"
    It's 32-bit Windows or a different processor type - see cause 3.
On a target PC: "Windows protected your PC" (SmartScreen)
    Normal for any app that isn't expensively code-signed: More info >
    Run anyway.
```

CHANGING THE CODE LATER:

Replace netscout.py / netscout_gui.py and run the matching build script again - netscout.spec stays unchanged.